Section 1: Multiprocessing

Parallellized Mandebrot Set Computation Times (L04)

```
| Workers | Time (s) | Speedup | Efficiency (%) |
|---------|----------|---------|----------------|
1 workers: 0.0712 s, speedup=1.00x, efficiency=100.0%
2 workers: 0.0407 s, speedup=1.75x, efficiency=87.5%
3 workers: 0.0544 s, speedup=1.31x, efficiency=43.6%
4 workers: 0.0448 s, speedup=1.59x, efficiency=39.7%
5 workers: 0.0425 s, speedup=1.68x, efficiency=33.5%
6 workers: 0.0394 s, speedup=1.81x, efficiency=30.1%
7 workers: 0.0311 s, speedup=2.29x, efficiency=32.7%
8 workers: 0.0328 s, speedup=2.17x, efficiency=27.2%
```

Chunk Sweeping L05

8 Workers:

```
Serial: 0.0620 s

| Chunks | Time (s) | Speedup | Efficiency (%) | LIF |
|--------|----------|---------|----------------|-----|
   8 chunks ( 1x workers): 0.0256 s, speedup=2.42x, efficiency=30.3%, LIF=2.302
  16 chunks ( 2x workers): 0.0157 s, speedup=3.95x, efficiency=49.4%, LIF=1.025
  32 chunks ( 4x workers): 0.0163 s, speedup=3.81x, efficiency=47.6%, LIF=1.102
  48 chunks ( 6x workers): 0.0145 s, speedup=4.29x, efficiency=53.6%, LIF=0.866
  64 chunks ( 8x workers): 0.0161 s, speedup=3.86x, efficiency=48.2%, LIF=1.075
 128 chunks (16x workers): 0.0178 s, speedup=3.48x, efficiency=43.5%, LIF=1.300
```

7 Workers:

```
Serial: 0.0619 s

| Chunks | Time (s) | Speedup | Efficiency (%) | LIF |
|--------|----------|---------|----------------|-----|
   7 chunks ( 1x workers): 0.0297 s, speedup=2.09x, efficiency=29.8%, LIF=2.353
  14 chunks ( 2x workers): 0.0179 s, speedup=3.47x, efficiency=49.5%, LIF=1.019
  28 chunks ( 4x workers): 0.0171 s, speedup=3.63x, efficiency=51.8%, LIF=0.931
  42 chunks ( 6x workers): 0.0149 s, speedup=4.15x, efficiency=59.3%, LIF=0.687
  56 chunks ( 8x workers): 0.0152 s, speedup=4.07x, efficiency=58.1%, LIF=0.722
 112 chunks (16x workers): 0.0159 s, speedup=3.89x, efficiency=55.5%, LIF=0.801
```

Full Performance Comparison at 1024x1024, max_iter=100

```
| Implementation | Time (s) | Speedup |
|----------------|----------|---------|
Naive Python     4.0889 s,   speedup=1.00x
Numpy Vectorized 0.9811 s,   speedup=4.17x
Numba (@njit)    0.0620 s,   speedup=65.95x
Multiprocessing  0.0145 s,   speedup=281.99x
Dask(6 workers, 32 chunks) 0.1276 s, speedup=32.04x
Dask(6 workers, 12 chunks) 0.0780 s, speedup=52.42x
```

Do the speedup results match what you expected?
Yes, the results matched my expectations. It shows we have an increase in speedup with an increasing number of chunks, however, when the chunk sizes become too small then the overhead starts to dominate and the speedup drops.

Section 2: Dask Local

Best Local Dask Implementation at 1024x1024

```
Serial baseline T1: 0.0670 s
6 workers:

n_chunks | time (s) | vs 1x | speedup | LIF
-----------------------------------------------
       6 |   0.1052 |  1.57x |    0.64x | 8.423
      12 |   0.0780 |  1.16x |    0.86x | 5.982
      24 |   0.0885 |  1.32x |    0.76x | 6.926
      36 |   0.1319 |  1.97x |    0.51x | 10.819
      48 |   0.1564 |  2.33x |    0.43x | 13.008
      96 |   0.2489 |  3.72x |    0.27x | 21.293

Record:
n_chunks_optimal = 12
t_min            = 0.0780 s
LIF_min          = 5.982
```

In multiprocessing, 6 workers was the most efficient hence the same configuration is tested in dask local. This used **cache=true**.

Is the scheduling overhead visible at small resolutions? Does performance improve as expected at larger N?
Dask is significantly slower than multiprocessing. This is indicated by the best Dask time of 0.0780 s with 12 chunks, which is still much slower than the multiprocessing time of 0.0145 s. The overhead of Dask's task scheduling and communication between workers contributes to

this performance gap, especially for a workload that may not be large enough to overrun these costs. The overhead difference suggests that multiprocessing is more suitable for workloads that require low-latency execution and have a smaller number of tasks, while Dask may be more appropriate for larger, more complex workloads that can benefit from its dynamic task scheduling and distributed computing capabilities. For smaller tasks or those with tight performance requirements, multiprocessing may be the better choice due to its lower overhead.

Section 3: Dask Distributed

Chunk Sweeps at different worker counts, 4096x4096, iter=100.

```
1 worker:
Serial baseline T1: 1.2107 s

n_chunks | time (s) | vs 1x | speedup | LIF
---------------------------------------------
       1 |   1.3915 |  1.15x |    0.87x | 0.149
       2 |   1.3819 |  1.14x |    0.88x | 0.141
       4 |   1.3873 |  1.15x |    0.87x | 0.146
       6 |   1.3855 |  1.14x |    0.87x | 0.144
       8 |   1.4147 |  1.17x |    0.86x | 0.168
      16 |   1.4230 |  1.18x |    0.85x | 0.175

Record:
n_chunks_optimal = 2
t_min            = 1.3819 s
LIF_min          = 0.141
```

```
2 workers:
Serial baseline T1: 1.2110 s

n_chunks | time (s) | vs 1x | speedup | LIF
---------------------------------------------
       2 |   0.9037 |  0.75x |    1.34x | 0.492
       4 |   0.8960 |  0.74x |    1.35x | 0.480
       8 |   1.0728 |  0.89x |    1.13x | 0.772
      12 |   0.9228 |  0.76x |    1.31x | 0.524
      16 |   1.0129 |  0.84x |    1.20x | 0.673
      32 |   0.9966 |  0.82x |    1.22x | 0.646

Record:
n_chunks_optimal = 4
t_min            = 0.8960 s
LIF_min          = 0.480
```

```
3 workers:
Serial baseline T1: 1.2305 s

n_chunks | time (s) | vs 1x | speedup | LIF
---------------------------------------------
       3 |   1.3057 |  1.06x |    0.94x | 2.183
       6 |   0.9078 |  0.74x |    1.36x | 1.213
      12 |   0.7955 |  0.65x |    1.55x | 0.940
      18 |   0.7054 |  0.57x |    1.74x | 0.720
      24 |   0.7200 |  0.59x |    1.71x | 0.756
      48 |   0.6820 |  0.55x |    1.80x | 0.663

Record:
n_chunks_optimal = 48
t_min            = 0.6820 s
LIF_min          = 0.663
```

```
4 workers:
Serial baseline T1: 1.2103 s

n_chunks | time (s) | vs 1x | speedup | LIF
---------------------------------------------
       4 |   0.8614 |  0.71x |    1.41x | 1.847
       8 |   0.8393 |  0.69x |    1.44x | 1.774
      16 |   0.6483 |  0.54x |    1.87x | 1.143
      24 |   0.5553 |  0.46x |    2.18x | 0.835
      32 |   0.5537 |  0.46x |    2.19x | 0.830
      64 |   0.5988 |  0.49x |    2.02x | 0.979

Record:
n_chunks_optimal = 32
t_min            = 0.5537 s
LIF_min          = 0.830
```

There wasn't a most optimal chunk count for the different amount of workers, however, for the best configuration of 4 workers, the optimal chunk count was 32. This configuration used 2 vCPUs and 4GB of ram. I ran 2 VMs which acted as workers.

12 Chunks, worker sweep:

```
Serial baseline: 1.2312s

| Chunks | Workers | Time (s) | Speedup |
|--------|---------|----------|---------|
12 chunks | 1 workers | 1.6306 s, speedup=0.75x
12 chunks | 2 workers | 0.9285 s, speedup=1.33x
12 chunks | 3 workers | 0.7495 s, speedup=1.64x
12 chunks | 4 workers | 0.6354 s, speedup=1.94x
```

Does speedup scale as expected with worker count? Any surprising results?
I chose 12 chunks for this experiment because it was the optimal chunk count for 4 workers at 1024x1024 resolution, and I wanted to see how the performance scaled with different worker counts at the larger 4096x4096 resolution. The results show that as the number of workers increases, the time decreases and the speedup increases, which is expected. However, the speedup is not linear with respect to the number of workers, indicating that there are diminishing returns as more workers are added, due to overhead.

Performance Table:

Speedup Table at 1024x1024 resolution, max_iter=100

```
| Implementation | Time (s) | Speedup |
|----------------|----------|---------|
Naive Python     4.0889 s,   speedup=1.00x
Numpy Vectorized 0.9811 s,   speedup=4.17x
Numba (@njit)    0.0620 s,   speedup=65.95x
Multiprocessing  0.0145 s,   speedup=281.99x
Dask(6 workers, 32 chunks) 0.1276 s, speedup=32.04x
Dask(6 workers, 12 chunks) 0.0780 s, speedup=52.42x
Dask Distributed (4 workers, 12 chunks) 0.0765 s, speedup=53.78x
```

The fact that Dask Distributed with 4 workers and 12 chunks achieves similar/slightly better performance than Dask local with 6 workers and 12 chunks suggests that the distributed machines may have better hardware capabilities (e.g. faster CPUs) that help offset the overhead of distributed execution. However, the performance is still much worse than multiprocessing, indicating that the overhead of Dask's distributed execution model is still a significant factor in this workload. It is also important to remember that this is at 1024x1024 resolution, meaning that the workload is not large enough to fully benefit from Dask's distributed capabilities, and the overhead may dominate the execution time. The Dask local implementation is with 6 workers on a 4 physical core and 8 logical core machine, which could be the reason 6 local workers performs worse than 4 distributed workers, as the local machine may be

oversubscribed with 6 workers, leading to increased contention and overhead, while the distributed setup had 1 worker per physical core, allowing for better performance.